

From Variational Inference to Diffusion-Based Reinforcement Learning

One objective, increasingly expressive samplers

Reward maximization

$$\max_{\theta} \mathbb{E}_q[R]$$

RL as inference

trajectory KL

DA-MDPs

diffusion RL



Variational inference

reverse KL

Diffusion samplers

path-space bound

LESSON 1

Reward Maximization

$$\max_{\theta} \mathbb{E}_{X \sim q_{\theta}}[R(X)]$$

Folder: L1-CostMin/ • Two unbiased gradient estimators

Lesson 1 folder: the weighted reward landscape

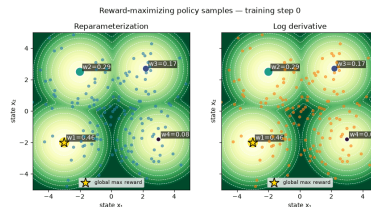
$$p_{\text{ref}}(x) = \sum_{i=1}^4 w_i \text{Normal}(x; \mu_i, \sigma^2 I),$$

$$R(x) = \log p_{\text{ref}}(x),$$

$$(w_1, w_2, w_3, w_4) = (0.46, 0.29, 0.17, 0.08),$$

$$\sum_i w_i = 1.$$

- ▶ w_i : probability mass of mode i
- ▶ Gold star: global reward maximum
- ▶ Yellow regions: high reward
- ▶ Gaussian policy: one mode only



Expected reward: the first objective

- **Policy:** diagonal Gaussian

$$q_{\theta}(x) = \mathcal{N}(x; \mu_{\theta}, \text{diag}(\sigma_{\theta}^2))$$

- **Objective:**

$$J(\theta) = \mathbb{E}_{q_{\theta}}[R(X)], \quad \theta \leftarrow \theta + \eta \nabla_{\theta} J(\theta)$$

- **PyTorch loss:** $L(\theta) = -J(\theta)$
- **Outcome:** one selected mode; vanishing variance

Reparameterization trick: move randomness outside

- **Fixed noise:**

$$\varepsilon \sim \mathcal{N}(0, I), \quad X = g_{\theta}(\varepsilon) = \mu_{\theta} + \sigma_{\theta} \odot \varepsilon$$

- **Pathwise gradient:**

$$\nabla_{\theta} J = \mathbb{E}_{\varepsilon} [\nabla_x R(g_{\theta}(\varepsilon)) \nabla_{\theta} g_{\theta}(\varepsilon)]$$

- **Monte Carlo loss:**

$$\hat{L}_{\text{path}} = -\frac{1}{N} \sum_{i=1}^N R(g_{\theta}(\varepsilon_i))$$

- Requires differentiable $R(x)$

Log-derivative trick: differentiate the density

- **Score identity:**

$$\nabla_{\theta} J = \mathbb{E}_{q_{\theta}}[R(X) \nabla_{\theta} \log q_{\theta}(X)]$$

- **Baseline:** zero expected contribution

$$\mathbb{E}_{q_{\theta}}[b \nabla_{\theta} \log q_{\theta}(X)] = 0$$

- **Surrogate loss:**

$$\hat{L}_{\text{score}} = -\frac{1}{N} \sum_i \text{stopgrad}(R_i - b_i) \log q_{\theta}(x_i)$$

- No reward gradient required

Leave-one-out baseline and estimator trade-offs

- ▶ **Leave-one-out baseline:** $b_i = \frac{1}{N-1} \sum_{j \neq i} R(x_j)$

Reparameterization

- ▶ `rsample`
- ▶ Gradient through x
- ▶ Requires $\nabla_x R$
- ▶ Lower variance

Log derivative

- ▶ `sample`
- ▶ Gradient through $\log q_\theta$
- ▶ No $\nabla_x R$
- ▶ Discrete variables supported

- ▶ Same expected gradient; different computation graph

Lesson 1: what you implement

- ▶ L1-CostMin/gaussian_policy_student.ipynb

A. reparameterization_loss

- ▶ `policy.distribution()`
- ▶ N samples via `rsample`
- ▶ Return negative mean reward

$$\hat{L}_{\text{path}} = -\frac{1}{N} \sum_i R(x_i).$$

- ▶ Run checks $\rightarrow$ train both policies $\rightarrow$ create GIF

B. log_derivative_loss

- ▶ Samples via `sample`
- ▶ Leave-one-out baseline b_i
- ▶ Detach $A_i = R_i - b_i$
- ▶ Return $-\text{mean}(A_i \log q_i)$

LESSON 2

Variational Inference

$$\min_{\theta} D_{\text{KL}}(q_{\theta} \| p_T) \iff \max_{\theta} (\mathbb{E}_q[R] + TH(q_{\theta}))$$

Folder: L2-MaxEntMin/ • Reward plus entropy

The recurring problem

- **Given:** evaluable reward $R(x)$

$$p_T(x) = \frac{e^{R(x)/T}}{Z_T}, \quad Z_T = \int e^{R(x)/T} dx$$

- **Learn:** sampler q_θ for reward and diversity

$$\max_{\theta} J_T(\theta) = \mathbb{E}_{q_\theta}[R(X)] + TH(q_\theta)$$

- **Progression:** richer sampler + tractable upper bound

- ▶ **Definition:** $D_{\text{KL}}(q\|p) = \mathbb{E}_{X \sim q} \left[\log \frac{q(X)}{p(X)} \right]$

Properties

- ▶ $D_{\text{KL}}(q\|p) \geq 0$
- ▶ $D_{\text{KL}}(q\|p) = 0 \iff q = p \text{ a.e.}$
- ▶ **Asymmetric:** $D_{\text{KL}}(q\|p) \neq D_{\text{KL}}(p\|q)$

Reverse KL intuition

- ▶ Samples come from q_θ
- ▶ Strong penalty where $p(x) \approx 0$
- ▶ Restricted q_θ : mode seeking

► **Reward-weighted target:**

$$\tilde{p}_T(x) = e^{R(x)/T}, \quad p_T(x) = \frac{\tilde{p}_T(x)}{Z_T}$$

► **Reverse KL:**

$$TD_{\text{KL}}(q_\theta \| p_T) = -(\mathbb{E}_q[R] + TH(q_\theta)) + T \log Z_T$$

► **Equivalent objective:**

$$\boxed{\min_{\theta} D_{\text{KL}}(q_\theta \| p_T) \iff \max_{\theta} (\mathbb{E}_q[R] + TH(q_\theta))}$$

Maximum entropy changes the sample cloud

► **Gaussian entropy:** $H(q_\theta) = \frac{1}{2} \sum_{d=1}^D \log(2\pi e \sigma_d^2)$

► **Pathwise loss:**

$$\hat{L}_{\text{path}} = -\frac{1}{N} \sum_i R(x_i) - TH(q_\theta)$$

► **Score signal:**

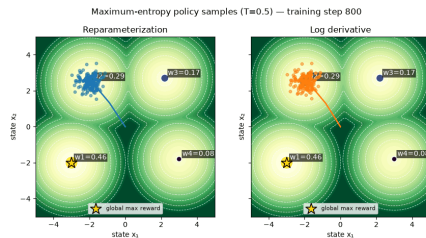
$$A_i = R(x_i) - T \log q_\theta(x_i) - b_i, \quad \hat{L}_{\text{score}} = -\frac{1}{N} \sum_i \text{stopgrad}(A_i) \log q_\theta(x_i)$$

► Entropy prevents variance collapse

Temperature: exploitation versus diversity

$$J_T(\theta) = \mathbb{E}_q[R] + TH(q),$$
$$p_T(x) \propto e^{R(x)/T}.$$

- ▶ $T \downarrow 0$: exploit reward maxima
- ▶ Moderate T : reward + finite spread
- ▶ Large T : entropy dominates



$T = 0.5$: visible sample spread

Lesson 2: what you implement

- ▶ L2-MaxEntMin/gaussian_policy_student.ipynb

A. reparameterization_maxent_loss

- ▶ States via `rsample`
- ▶ Mean reward
- ▶ Exact Gaussian entropy
- ▶ Return $-\bar{R} - TH(q_\theta)$

$$\hat{L}_{\text{path}} = -\hat{J}_T.$$

- ▶ Verify: $J_T \uparrow$, $TD_{\text{KL}} \downarrow$, finite sample spread

B. log_derivative_maxent_loss

- ▶ `sample` and `log_prob`
- ▶ $S_i = R_i - T \log q_\theta(x_i)$
- ▶ Leave-one-out centering; `detach`
- ▶ Return $-\text{mean}(\text{stopgrad}(S_i - b_i) \log q_i)$

LESSON 3

RL as Variational Inference

$$\min_{\theta} D_{\text{KL}}(q_{\theta}(\tau) \| p_T(\tau)) \iff \max_{\theta} J_{\text{ME}}(\theta)$$

Theory only • From reward-weighted samples to reward-weighted trajectories

A policy induces a distribution over trajectories

- ▶ **Trajectory:** $\tau = (s_0, a_0, \dots, a_H, s_{H+1})$
- ▶ **Policy-induced distribution:**

$$q_{\theta}(\tau) = p(s_0) \prod_{t=0}^H q_{\theta}(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t)$$

- ▶ **Reward-weighted target:**

$$\tilde{p}_T(\tau) = p(s_0) \prod_{t=0}^H p(s_{t+1} \mid s_t, a_t) e^{R_{\text{env}}(s_t, a_t)/T}$$

- ▶ Same VI template; trajectories replace samples

First DPI: actions to state–action trajectories

- ▶ **Intractable marginal:** $q_{\theta}(a_{0:H}) = \int q_{\theta}(a_{0:H}, s_{0:H+1}) ds$
- ▶ **Marginalization is a channel:**

$$D_{\text{KL}}(q_{\theta}(a_{0:H}) \| p_T(a_{0:H})) \leq D_{\text{KL}}(q_{\theta}(a_{0:H}, s_{0:H+1}) \| p_T(a_{0:H}, s_{0:H+1}))$$

- ▶ Rollouts expose every (s_t, a_t, s_{t+1})
- ▶ Shared dynamics cancel in the KL ratio

Reverse KL becomes maximum-entropy RL

- ▶ **Joint reverse KL:**

$$TD_{\text{KL}}(q_{\theta}(\tau) \| p_T(\tau)) = -J_{\text{ME}}(\theta) + T \log Z_T$$

- ▶ **MaxEnt RL objective:**

$$J_{\text{ME}}(\theta) = \mathbb{E}_{q_{\theta}} \sum_{t=0}^H [R_{\text{env}}(s_t, a_t) - T \log q_{\theta}(a_t \mid s_t)]$$

- ▶ Reward: exploitation
- ▶ Policy entropy: exploration
- ▶ MaxEnt RL = VI over trajectories

The local variational target produces policy improvement

- ▶ **Local Boltzmann target:**

$$p^*(a \mid s) = \frac{\exp(Q^{q_{\theta_*}}(s, a)/T)}{Z(s)}$$

- ▶ **State-conditional VI update:**

$$L_{\text{ME}}(\theta) = T \sum_t \mathbb{E}_{s_t \sim q_{\theta_*}} D_{\text{KL}}(q_{\theta}(a_t \mid s_t) \parallel p^*(a_t \mid s_t))$$

- ▶ **Soft value:** $V^q(s) = \mathbb{E}_{a \sim q}[Q^q(s, a) - T \log q(a \mid s)]$

- ▶ Trust region: keep θ near θ_*

Lesson 3 map: variational inference becomes control

- ▶ Sample $x \rightarrow$ trajectory τ
- ▶ $q_\theta(x) \rightarrow$ policy-induced $q_\theta(\tau)$
- ▶ Reward $R(x) \rightarrow$ return $\sum_t R_{\text{env}}$
- ▶ Boltzmann target $\rightarrow$ reward-weighted trajectories
- ▶ Entropy bonus $\rightarrow$ causal policy entropy
- ▶ Reverse KL $\rightarrow$ MaxEnt RL

- ▶ **Theory only:** no new code
- ▶ **Bridge:** distributions $\rightarrow$ trajectories

LESSON 4

Diffusion Samplers

$N(0, I) \longrightarrow q_\theta(x^0)$ with many modes

Folder: L4-DiffusionSamplers/ • Learn a reverse stochastic process

Why move beyond a Gaussian?

- ▶ Target: multimodal
- ▶ Gaussian policy: unimodal
- ▶ Reverse KL: mode seeking
- ▶ Entropy: wider mode, not more modes

Desired sampler

- ▶ Simple prior
- ▶ Many conditional transitions
- ▶ Multimodal endpoint

$$X^K \sim \mathcal{N}(0, I) \longrightarrow X^{K-1} \longrightarrow \dots \longrightarrow X^0.$$

Diffusion models: noise out, structure back in

Forward process: fixed

- ▶ Add Gaussian noise
- ▶ Known transitions

$$x^0 \longrightarrow x^1 \longrightarrow \dots \longrightarrow x^K \approx \mathcal{N}(0, I).$$

Reverse process: learned

- ▶ Start from noise
- ▶ Learn denoising transitions

$$x^K \longrightarrow x^{K-1} \longrightarrow \dots \longrightarrow x^0 \sim q_\theta.$$

- ▶ Gaussian steps; multimodal composition

- ▶ **VP-SDE:**

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dW_t$$

- ▶ **Forward kernel:** $p(x^k | x^{k-1}) = \mathcal{N}(\sqrt{\alpha_k}x^{k-1}, \beta_k I)$

- ▶ **Reverse kernel:**

$$q_\theta(x^{k-1} | x^k) = \mathcal{N}\left(\frac{x^k + \beta_k s_\theta(x^k, k)}{\sqrt{\alpha_k}}, \beta_k I\right)$$

- ▶ $q_K = \mathcal{N}(0, I)$; $K = 24$; $\beta_k : 0.02 \rightarrow 0.18$

- ▶ Schedule flag: fixed or learned monotone interior points

- ▶ **DPI for channel M :** $D_{\text{KL}}(qM \| pM) \leq D_{\text{KL}}(q \| p)$
- ▶ **KL chain rule:**

$$D_{\text{KL}}(q(x, z) \| p(x, z)) = D_{\text{KL}}(q(x) \| p(x)) + \mathbb{E}_{q(x)} D_{\text{KL}}(q(z | x) \| p(z | x))$$

- ▶ Marginal density $q_\theta(x^0)$: intractable
- ▶ Path transition densities: tractable
- ▶ Joint path KL: upper bound on endpoint KL

The diffusion path-space upper bound

► Reverse and forward paths:

$$q_{\theta}(x^{0:K}) = q_K(x^K) \prod_{k=1}^K q_{\theta}(x^{k-1} \mid x^k), \quad p_T(x^{0:K}) = p_T(x^0) \prod_{k=1}^K p(x^k \mid x^{k-1})$$

► DPI upper bound:

$$TD_{\text{KL}}(q_{\theta}(x^0) \| p_T(x^0)) \leq TD_{\text{KL}}(q_{\theta}(x^{0:K}) \| p_T(x^{0:K}))$$

► Trainable loss, up to $T \log Z_T$:

$$L_{\text{path}} = \mathbb{E}_{q_{\theta}} \left[-R(x^0) + T \log q_K(x^K) + T \sum_{k=1}^K \log \frac{q_{\theta}(x^{k-1} \mid x^k)}{p(x^k \mid x^{k-1})} \right]$$

Optional Langevin preconditioning and clipping

- ▶ **Target score:**

$$g_T(x) = \text{clip} \left(\frac{\nabla_x R(x)}{T}, -10^2, 10^2 \right)$$

- ▶ **Preconditioned drift (Vargas et al., 2023):**

$$f_\theta(k, x) = \text{clip}(\text{NN}_1(k, x) + \text{NN}_2(k) \odot g_T(x), -10^4, 10^4)$$

- ▶ **Reverse score:** $s_\theta(k, x) = -x + f_\theta(k, x)$
- ▶ Detached analytic gradient
- ▶ Score and drift clipping
- ▶ Flag: `USE_LANGEVIN_PRECONDITIONING`

Lesson 4: what you implement

- ▶ L4-DiffusionSamplers/diffusion_sampler_student.ipynb

A. Forward process

- ▶ forward_sde_step

$$x^k = \sqrt{1 - \beta_k} x^{k-1} + \sqrt{\beta_k} \varepsilon.$$

- ▶ forward_kernel_log_prob

$$\log p(x^k | x^{k-1}).$$

B. Learned reverse process

- ▶ reverse_sde_step using s_θ

- ▶ reverse_kernel_log_prob

$$\log q_\theta(x^{k-1} | x^k).$$

- ▶ Match Gaussian parameters in sampling and log probability

Lesson 4: what the exercise already provides

Optimization

- ▶ time-conditioned score network;
- ▶ reparameterization path loss;
- ▶ leave-one-out log-derivative loss;
- ▶ $T = 2 \rightarrow 1$ mode curriculum

Experiment controls

- ▶ GRADIENT_ESTIMATOR;
- ▶ LEARN_DIFFUSION_SCHEDULE;
- ▶ USE_LANGEVIN_PRECONDITIONING;
- ▶ 600 samples; one frame per second

- ▶ $N(0, I) \xrightarrow{24 \text{ reverse steps}}$ weighted multimodal samples
- ▶ Fixed GIF noise: each dot has a visible trajectory

LESSON 5

Diffusion-Augmented MDPs

$(t, k) \mapsto \tilde{t}$ turns denoising into control

A second DPI • One augmented state per diffusion step

Diffusion policy: one action becomes a denoising trajectory

- ▶ **Prior action:** $a_t^K \sim q_{\text{prior}}$
- ▶ **Conditional denoising:**

$$a_t^{k-1} \sim q_{\theta}(a_t^{k-1} \mid a_t^k, s_t), \quad k = K, \dots, 1$$

- ▶ **Executed action:** a_t^0
- ▶ **Implicit policy:**

$$q_{\theta}(a_t^0 \mid s_t) = \int q_{\text{prior}}(a_t^K) \prod_{k=1}^K q_{\theta}(a_t^{k-1} \mid a_t^k, s_t) da_t^{1:K}$$

- ▶ Multimodal samples; intractable marginal density

Second DPI: include every diffusion latent

- ▶ **Second DPI:**

$$D_{\text{KL}}(q_{\theta}(a_{0:H}, s) \| p_T(a_{0:H}, s)) \leq D_{\text{KL}}(q_{\theta}(a_{0:H}^{0:K}, s) \| p_T(a_{0:H}^{0:K}, s))$$

- ▶ **Upper-bound decomposition:**

$$\mathbb{E} \sum_{t=0}^H \sum_{k=1}^K \left[T \log \frac{q_{\theta}(a_t^{k-1} | a_t^k, s_t)}{p(a_t^k | a_t^{k-1}, s_t)} - \mathbf{1}_{\{k=1\}} R_{\text{env}}(s_t, a_t^0) \right]$$

- ▶ Local, evaluable transition factors

Flatten two clocks into a diffusion-augmented MDP

► Flattened time:

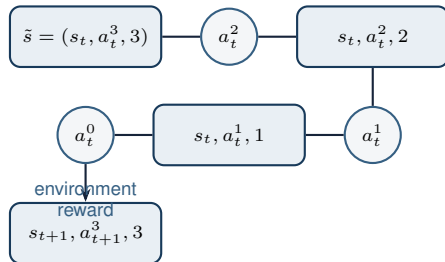
$$\tilde{t}(t, k) = tK + (K - k),$$

$$\tilde{s}_{\tilde{t}} = (s_t, a_t^k, k),$$

$$\tilde{a}_{\tilde{t}} = a_t^{k-1}.$$

► Delayed environment reward:

$$\tilde{R}(\tilde{s}, \tilde{a}) = \begin{cases} 0, & k > 1, \\ R_{\text{env}}(s_t, a_t^0), & k = 1. \end{cases}$$



► **Augmented soft reward:**

$$r_{\text{soft}}^{t,k} = \mathbf{1}_{\{k=1\}} R_{\text{env}}(s_t, a_t^0) - T \log \frac{q_{\theta}(a_t^{k-1} \mid a_t^k, s_t)}{p(a_t^k \mid a_t^{k-1}, s_t)}$$

► **Soft value recursion:**

$$V_{\text{DA}}(\tilde{s}) = \mathbb{E}_{q_{\theta}} \left[Q_{\text{DA}}(\tilde{s}, \tilde{a}) - T \log \frac{q_{\theta}(\tilde{a} \mid \tilde{s})}{p(a^k \mid \tilde{a}, s)} \right]$$

► Forward/reverse log ratio replaces policy entropy

A local DA-MDP policy update

- ▶ **Local target:**

$$p^*(a^{k-1} \mid \tilde{s}) \propto p(a^k \mid a^{k-1}, s) \exp(Q_{\text{DA}}(s, a^{k-1})/T)$$

- ▶ **Surrogate update:**

$$\mathbb{E}_{\tilde{s} \sim q_{\theta_*}} D_{\text{KL}}(q_{\theta}(a^{k-1} \mid \tilde{s}) \parallel p^*(a^{k-1} \mid \tilde{s}))$$

- ▶ One reverse step per augmented state
- ▶ No full-chain backpropagation
- ▶ PPO, REPPPO, or WPO variants

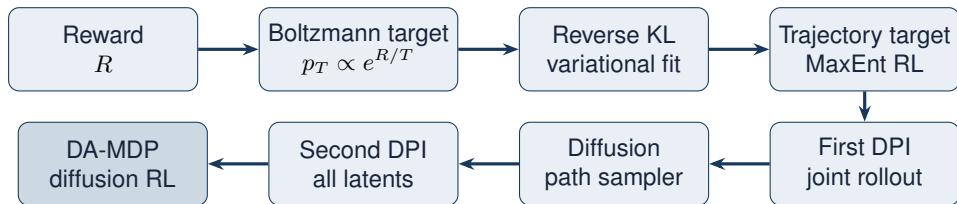
DA-MDP rollout and update loop

- ▶ Sample prior noise a_t^K at state s_t
- ▶ Denoise $a_t^K \rightarrow \dots \rightarrow a_t^0$
- ▶ Store every augmented transition
- ▶ Execute a_t^0 ; observe reward and s_{t+1}
- ▶ Compute augmented returns and advantages
- ▶ Apply a constrained on-policy update

Paper result

- ▶ PPO, REPPO, and WPO variants
- ▶ Standard continuous-control performance
- ▶ Multimodal action distributions

One mathematical story



- ▶ One objective: fit reward-weighted targets
- ▶ Increasingly expressive distributions
- ▶ Two DPIs make implicit policies tractable

References and course material

- ▶ Sanokowski & Patil (2026): *Diffusion-Augmented MDPs*
- ▶ Kingma & Welling (2014): *Auto-Encoding Variational Bayes*
- ▶ Williams (1992): *REINFORCE*
- ▶ Song et al. (2021): *Score-Based Generative Modeling*
- ▶ Vargas, Grathwohl & Doucet (2023): *Denoising Diffusion Samplers*
- ▶ Levine (2018): *RL and Control as Probabilistic Inference*
- ▶ Course folders: L1-CostMin/ → L2-MaxEntMin/ → L4-DiffusionSamplers/